

Aegis AI — Complete API Test Guide

Every Feature, Every Edge Case

Setup Before Testing

Ensure the API is running:

```
powershell  
  
docker-compose up --build  
# Wait for: "Application startup complete."
```

Open Swagger UI: <http://localhost:8000/docs>

Base URL for all tests: <http://localhost:8000>

TEST GROUP 1 — Schema & Input Validation

These tests verify the API accepts all valid inputs and rejects invalid ones correctly.

Test 1.1 — Full URL with https

```
json  
  
POST /scan  
{ "url": "https://google.com" }
```

Expected: 200, [risk_level: "Safe"](#), [confidence_score < 10](#)

Test 1.2 — URL without scheme (NEW BUG FIX)

```
json  
  
POST /scan  
{ "url": "google.com" }
```

Expected: 200 (NOT 422). The API auto-prepends <https://google.com>. Previously returned 422 Unprocessable Entity — this is now fixed.

Test 1.3 — www prefix without scheme

```
json  
  
POST /scan  
{ "url": "www.google.com" }
```

Expected: 200. Normalized to `https://www.google.com`.

Test 1.4 — HTTP URL

```
json  
  
POST /scan  
{ "url": "http://example.com" }
```

Expected: 200, `is_http` flag raised, `ssl_invalid: true` in ML features.

Test 1.5 — Empty URL

```
json  
  
POST /scan  
{ "url": "" }
```

Expected: 422 Unprocessable Entity, detail: `URL cannot be empty`

Test 1.6 — Completely invalid input

```
json  
  
POST /scan  
{ "url": "not_a_url!!@# $" }
```

Expected: 200 (API will auto-prepend https and attempt scan — scan will fail gracefully with error flags)

Test 1.7 — Missing url field

```
json  
  
POST /scan  
{ }
```

Expected: 422 Unprocessable Entity, `field required`

TEST GROUP 2 — Core Scan Results (Risk Classification)

Test 2.1 — Known Safe URL

```
json

POST /scan
{ "url": "https://google.com" }
```

Expected:

- `risk_level: "Safe"`
- `confidence_score < 5`
- `heuristics.is_suspicious: false`
- `whois.domain_age_days > 10000` (Google is 28 years old)
- `ssl.details.is_trusted_major_ca: true`
- `redirects.is_www_normalization: true` (google.com → www.google.com is www redirect, not a threat)
- `total_flags: 0` or 1 (only DNS score slight flag)
- VT: `malicious: 0`

Test 2.2 — Known Phishing URL

```
json

POST /scan
{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

Expected:

- `risk_level: "High Risk"`
- `confidence_score: 90`
- `heuristics.flags` includes: suspicious TLD, typosquatting, phishing keywords
- `dns.details.resolves: false` (domain is taken down)
- `ssl.details.is_valid: false` (HTTP, no SSL)
- `virustotal.detection_counts.malicious >= 3`
- `ml_prediction.prediction: 1` (phishing)
- `ml_prediction.phishing_probability > 70`
- `total_flags >= 10`

Test 2.3 — GitHub (legit HTTPS, no MX)

json

POST /scan

```
{ "url": "https://github.com" }
```

Expected:

- `risk_level: "Safe"`
- `confidence_score < 5`
- `dns.details.has_mx: false` — GitHub doesn't use github.com for email, this should NOT be a critical signal
- `ssl.details.is_trusted_major_ca: true` (DigiCert)
- `whois.domain_age_days > 6000`

Test 2.4 — URL Shortener (Medium Risk)

json

POST /scan

```
{ "url": "https://bit.ly/3example" }
```

Expected:

- `risk_level: "Medium Risk"` or `"Low Risk"`
- `redirects.shorteners_found` contains "bit.ly"
- `redirects.hop_count >= 2`
- `redirect_score >= 40`
- At least 1 redirect flag raised

Test 2.5 — Raw IP URL (suspicious)

json

POST /scan

```
{ "url": "http://192.168.1.1/admin" }
```

Expected:

- `risk_level: "Medium Risk"` or higher
- `heuristics.flags` includes IP address flag

- `is_http: 1.0` in ML features
- `has_ip_address: 1.0` in ML features

Test 2.6 — High-entropy domain

```
json

POST /scan
{ "url": "https://xn--e1afmkfd.xn--plai" }
```

Expected:

- Entropy flag may fire
- Domain correctly processed without errors

Test 2.7 — claude.ai (known minor false positive)

```
json

POST /scan
{ "url": "https://claude.ai" }
```

Expected:

- `risk_level: "Safe"` ✓ (overall classification correct)
- `confidence_score: ~2-3` ✓
- `dns.flags` contains "No MX records" — this is a known limitation (claude.ai is a product site, not an email domain, but DNS check flags no MX)
- `ssl.details.is_free_cert: true` — Let's Encrypt flag fires (another limitation)
- `ml_prediction.ml_risk_level: "Low Risk"` with ~30% phishing probability — ML false positive due to limited training data. Improve by submitting feedback: `POST /feedback { feedback_type: "false_positive", corrected_risk: "Safe" }`
- `screenshot_url` should now be an English-language screenshot (US locale fix applied)

TEST GROUP 3 — Heuristics Engine (14 Checks)

Tests to verify each heuristic check fires correctly.

Test 3.1 — Phishing keywords

```
json  
  
POST /scan  
{ "url": "https://secure-account-verify-login.com/confirm" }
```

Expected flag: Phishing keywords in URL: [login, verify, secure, account, confirm]

Test 3.2 — Suspicious TLD

```
json  
  
POST /scan  
{ "url": "https://normaldomain.xyz" }
```

Expected flag: Suspicious TLD '.xyz'

Test 3.3 — Typosquatting detection

```
json  
  
POST /scan  
{ "url": "https://g00gle.com" }
```

Expected flags:

- Possible typosquatting detected: Very similar to 'google' (edit distance: 2) ☒ (heuristics fire)
- NXDOMAIN flag (domain doesn't resolve)

Expected overall result: High Risk 90%

⚠ IMPORTANT: The test verifies the typosquatting flag fires (which it does). The High Risk 90% is **CORRECT** — g00gle.com has **5 VT malicious detections** triggering the malicious ≥ 5 → score ≥ 90 override. This is intended behaviour — any user receiving a g00gle.com link should be blocked regardless of who registered the domain. WHOIS shows Google registered it defensively (MarkMonitor, 9,037 days old) but that doesn't reduce the danger of sharing the link.

Test 3.4 — IP address in URL

```
json  
  
POST /scan  
{ "url": "http://142.250.80.46/search" }
```

Expected flag: IP address flag in heuristics

Test 3.5 — Long URL

json

POST /scan

```
{ "url": "https://legitimate-looking-site.com/very/long/path/that/goes/on/and/on/with/lots/of/query?param1=value1&param2=
```

Expected flag: Excessive URL length flag, possible open redirect flag

Test 3.6 — @ symbol in URL

json

POST /scan

```
{ "url": "https://google.com@evil.com/path" }
```

Expected flag: (@ symbol in URL) — browser will navigate to evil.com, not google.com

TEST GROUP 4 — WHOIS Analysis

Test 4.1 — Very new domain

json

POST /scan

```
{ "url": "https://brandnewdomain123456789.com" }
```

Expected: (whois.domain_age_days < 30), WHOIS flag fires for new domain

Test 4.2 — Old established domain

json

POST /scan

```
{ "url": "https://microsoft.com" }
```

Expected: (whois.domain_age_days > 10000), no WHOIS flags, (whois_score: 0)

Test 4.3 — WHOIS unavailable

json

```
POST /scan
```

```
{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

Expected: `whois.creation_date: null`, WHOIS privacy flag fires

TEST GROUP 5 — DNS Analysis

Test 5.1 — NXDOMAIN (non-existent domain)

```
json
```

```
POST /scan
```

```
{ "url": "https://thisdomain-absolutely-does-not-exist-123456.com" }
```

Expected:

- `dns.details.resolves: false`
- `dns.flags` includes NXDOMAIN flag
- `dns_score > 40`

Test 5.2 — Domain with no MX

```
json
```

```
POST /scan
```

```
{ "url": "https://claud.ai" }
```

Expected: `dns.details.has_mx: false`, DNS MX flag fires

Test 5.3 — Domain with full DNS health

```
json
```

```
POST /scan
```

```
{ "url": "https://google.com" }
```

Expected:

- `dns.details.has_mx: true`
- `dns.details.has_spf: true`
- `dns.details.nameservers` has 4+ entries

- `dns_score < 10`
-

TEST GROUP 6 — SSL/TLS Analysis

Test 6.1 — No SSL (HTTP)

```
json  
  
POST /scan  
{ "url": "http://example.com" }
```

Expected:

- `ssl.flags` includes HTTP no-encryption flag
- `ssl.details.is_valid: false`
- `ssl_score > 0`

Test 6.2 — Let's Encrypt cert (legitimate)

```
json  
  
POST /scan  
{ "url": "https://claude.ai" }
```

Expected:

- `ssl.details.is_free_cert: true`
- `ssl.details.issuer: "Let's Encrypt"`
- `ssl_score: 10` (minor flag, not critical)
- `ssl.is_suspicious: false` (should NOT be marked suspicious for Let's Encrypt alone)

Test 6.3 — Trusted major CA

```
json  
  
POST /scan  
{ "url": "https://google.com" }
```

Expected:

- `ssl.details.is_trusted_major_ca: true`

- `ssl.details.tls_protocol: "TLSv1.3"`
 - `ssl_score: 0`
-

TEST GROUP 7 — Redirect Chain Analysis

Test 7.1 — WWW normalization (should NOT be flagged)

```
json

POST /scan
{ "url": "https://google.com" }
```

Expected:

- `redirects.hop_count: 2`
- `redirects.is_www_normalization: true`
- `redirects.redirect_score: 0` (www redirect is NOT suspicious)
- No redirect flags

Test 7.2 — URL shortener redirect

```
json

POST /scan
{ "url": "https://tinyurl.com/example" }
```

Expected:

- `redirects.shorteners_found` non-empty
- `has_shortener` flag fires
- `redirect_score > 30`

Test 7.3 — No redirect (direct connection)

```
json

POST /scan
{ "url": "https://github.com" }
```

Expected:

- `hop_count: 1` (direct 200)
 - `destination_changed: false`
 - `redirect_score: 0`
-

TEST GROUP 8 — Threat Feed Integration

Test 8.1 — URLhaus lookup (clean URL)

```
json  
  
POST /scan  
{ "url": "https://google.com" }
```

Expected:

- `urlhaus.found: false`
- `urlhaus.status: "not_found"`
- `urlhaus_score: 0`

Test 8.2 — OpenPhish feed loaded

```
json  
  
POST /scan  
{ "url": "https://google.com" }
```

Expected:

- `phishtank.feed_size > 100` — if 0, the feed is not loading (check network)
- `phishtank.source: "openphish"`
- `phishtank.found: false` for Google

Test 8.3 — GSB API available

```
json  
  
POST /scan  
{ "url": "https://google.com" }
```

Expected:

- `gsb.api_available: true`
 - `gsb.found: false`
 - `gsb_score: 0`
-

TEST GROUP 9 — VirusTotal Integration

Test 9.1 — Safe URL VT result

```
json
POST /scan
{ "url": "https://github.com" }
```

Expected:

- `detection_counts.malicious: 0`
- `detection_counts.harmless > 50`
- `scanners_count: 94`
- `virustotal_report` URL is valid

Test 9.2 — Phishing URL VT result

```
json
POST /scan
{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

Expected:

- `detection_counts.malicious >= 3` (triggers critical signal)
 - VT score contributes `30` weighted points to `virustotal` breakdown
-

TEST GROUP 10 — ML Classifier

Test 10.1 — ML prediction on known phishing

```
json
```

POST /scan

```
{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

Expected:

- `ml_prediction.available: true`
- `ml_prediction.prediction: 1` (phishing)
- `ml_prediction.ml_risk_level: "High Risk"`
- `ml_prediction.phishing_probability > 70`
- `top_features` shows `ssl_invalid`, `has_phishing_keywords`, `is_http`, `suspicious_tld`

Test 10.2 — ML prediction on safe URL

json

POST /scan

```
{ "url": "https://google.com" }
```

Expected:

- `ml_prediction.prediction: 0` (benign)
- `ml_prediction.ml_risk_level: "Safe"`
- `ml_prediction.phishing_probability < 20`

Test 10.3 — ML unavailable (before model.pkl placed)

Before placing `model.pkl` in `app/ml/`:

json

POST /scan

```
{ "url": "https://google.com" }
```

Expected:

- `ml_prediction.available: false`
- `ml_prediction.error: "Model file not found..."`
- Main scan STILL completes — ML is an optional layer

Test 10.4 — Feature vector inspection

```
json

POST /scan

{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

Check `ml_prediction.top_features` contains these for this URL:

- `ssl_invalid: 1.0` (HTTP = no SSL)
 - `has_phishing_keywords: 1.0` (login, verify, secure, account, confirm)
 - `is_http: 1.0` (no HTTPS)
 - `suspicious_tld: 1.0` (.tk)
 - `entropy: 0.788` (high entropy domain)
-

TEST GROUP 11 — Bulk Scan

Test 11.1 — Mixed batch

```
json

POST /scan/bulk

{
  "urls": [
    "https://google.com",
    "http://paypal-secure-verify-account.tk/login/confirm",
    "https://github.com"
  ]
}
```

Expected:

- `total_urls: 3`
- `completed: 3`
- `failed: 0`
- `highest_risk_url: "http://paypal-secure-verify-account.tk/login/confirm"`
- `highest_risk_level: "High Risk"`
- Google and GitHub both `risk_level: "Safe"`
- All 3 run concurrently (total time $\approx$ slowest individual scan)

Test 11.2 — URLs without scheme in bulk

```
json

POST /scan/bulk

{
  "urls": [
    "google.com",
    "github.com"
  ]
}
```

Expected: 200 — both auto-normalized to `https://google.com`, `https://github.com`

Test 11.3 — Exceeds limit

```
json

POST /scan/bulk

{
  "urls": ["https://a.com", "https://b.com", "https://c.com", "https://d.com", "https://e.com",
    "https://f.com", "https://g.com", "https://h.com", "https://i.com", "https://j.com",
    "https://k.com"]
}
```

Expected: 400 or 422, `Maximum 10 URLs per bulk scan request`

Test 11.4 — Empty urls array

```
json

POST /scan/bulk

{ "urls": [] }
```

Expected: 422, `At least 1 URL is required`

Test 11.5 — Single URL in bulk

```
json

POST /scan/bulk

{ "urls": ["https://google.com"] }
```

Expected: 200, works fine, `total_urls: 1`

TEST GROUP 12 — Async Scan

Test 12.1 — Submit and poll (full flow)

Step 1 — Submit:

```
json

POST /scan/async
{ "url": "https://google.com" }
```

Expected:

- Response in < 200ms
- `status: "pending"`
- `job_id` is a UUID string
- `poll_url: "/scan/status/{job_id}"`

Step 2 — Poll immediately:

```
GET /scan/status/{job_id}
```

Expected: `status: "pending"` or `"running"`

Step 3 — Poll after 60–90 seconds:

```
GET /scan/status/{job_id}
```

Expected:

- `status: "complete"`
- `result` contains full scan result
- `elapsed_seconds` shows duration

Test 12.2 — Invalid job_id

```
GET /scan/status/00000000-0000-0000-0000-000000000000
```

Expected: 404, `Scan job '...' not found or expired`

Test 12.3 — Async scan for high-risk URL

json

POST /scan/async

```
{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

Expected after polling:

- `result.risk_level: "High Risk"`
 - All flags present in `result`
-

TEST GROUP 13 — Feedback Loop

Test 13.1 — Submit a false positive correction

json

POST /feedback

```
{  
  "scan_id": "u-9d116b1b...",  
  "url": "https://google.com",  
  "original_risk": "Low Risk",  
  "corrected_risk": "Safe",  
  "feedback_type": "false_positive",  
  "user_note": "Google is clearly safe",  
  "confidence_score": 0.4,  
  "total_flags": 1  
}
```

Expected:

- `status: "received"`
- `feedback_id` (integer, increments)
- `submitted_at` timestamp

Test 13.2 — Submit a correct confirmation

json

POST /feedback

```
{
  "scan_id": "u-e00e0272...",
  "url": "http://paypal-secure-verify-account.tk/login/confirm",
  "original_risk": "High Risk",
  "corrected_risk": "High Risk",
  "feedback_type": "correct",
  "user_note": "Confirmed phishing site",
  "confidence_score": 90.0,
  "total_flags": 12
}
```

Expected: 200, `status: "received"`

Test 13.3 — View feedback statistics

GET /feedback/stats

Expected:

- `total_feedback >= 2` (after tests 13.1 and 13.2)
- `breakdown_by_type` shows `false_positive: 1`, `correct: 1`
- `training_ready: false` (until 50 samples)

Test 13.4 — All feedback_type values

Test each type: `false_positive`, `false_negative`, `wrong_level`, `correct`

TEST GROUP 14 — Metrics & Observability

Test 14.1 — JSON metrics

GET /metrics

Expected:

- `summary.total_scans > 0` (after running other tests)
- `summary.cache_hit_rate_pct` shows percentage
- `summary.avg_scan_time_ms` in milliseconds
- `risk_distribution` shows counts per level

- `daily_scans_last_7_days` shows today's count

Test 14.2 — Prometheus metrics

```
GET /metrics/prometheus
```

Expected: Plain text response, format:

```
# HELP aegis_total_scans Total number of URLs scanned
# TYPE aegis_total_scans counter
aegis_total_scans 15
...
```

Content-Type should be `text/plain; version=0.0.4`

Test 14.3 — Cache hit behavior

Scan the same URL twice:

```
json

POST /scan
{ "url": "https://google.com" }
```

Run twice. Second response should be faster (< 1 second, cache hit). Check `GET /metrics` — `cache_hits` increments on second call.

TEST GROUP 15 — Screenshot Proxy (Bug Fix)

Test 15.1 — Valid screenshot URL

First scan a URL that completes (after VT+URLScan polling):

```
json

POST /scan
{ "url": "https://claude.ai" }
```

Get the `screenshot_url` from the response (e.g., `https://urlscan.io/screenshots/xxx.png`).

Then proxy it:

```
GET /proxy-image?url=https://urlscan.io/screenshots/019ca1c5-6bc6-735f-b522-e812b90c6bb1.png
```

Expected: Actual image bytes returned, `Content-Type: image/png` The screenshot should now show the **English version** of the page (US locale fix applied).

Test 15.2 — Invalid/expired screenshot URL

```
GET /proxy-image?url=https://urlscan.io/screenshots/00000000-0000-0000-0000-000000000000.png
```

Expected: 404, `{"detail": "Image not found at source URL"}`

Test 15.3 — Missing URL parameter

```
GET /proxy-image
```

Expected: 422, `field required`

TEST GROUP 16 — Rate Limiting

Test 16.1 — POST /scan (5 per minute)

Send 6 requests rapidly:

```
bash

for i in {1..6}; do
  curl -s -X POST http://localhost:8000/scan \
    -H "Content-Type: application/json" \
    -d '{"url": "https://google.com"}' | python -m json.tool
done
```

Expected: First 5 succeed (200), 6th returns 429 Too Many Requests.

Test 16.2 — POST /scan/bulk (2 per minute)

Send 3 bulk requests rapidly. **Expected:** First 2 succeed, 3rd returns 429.

TEST GROUP 17 — Health & Service Endpoints

Test 17.1 — Root endpoint

GET /

Expected:

```
json
{
  "service": "Aegis Link Analyzer",
  "version": "4.0.0",
  "status": "running",
  "docs": "/docs",
  "metrics": "/metrics"
}
```

Test 17.2 — Health check

GET /health

Expected:

```
json
{
  "status": "healthy",
  "ml_model": "loaded",
  "timestamp": "2026-02-28T..."
}
```

`ml_model` is `"unavailable"` if `model.pkl` is not placed yet.

Test 17.3 — Swagger UI

Open: `http://localhost:8000/docs` **Expected:** Professional Swagger UI with all endpoints, no "Phase 3" or "Phase 4" labels in descriptions.

TEST GROUP 18 — Edge Cases & Error Handling

Test 18.1 — Very long URL

json

POST /scan

[illegible]

Expected: 200, URL processes without crashing.

Test 18.2 — Non-existent domain scan

json

POST /scan

```
{ "url": "https://this-domain-does-not-exist-aegis-test-xyz.com" }
```

Expected:

- `dns.details.resolves: false`
- `redirects.flags` includes connection failure
- Still returns 200 with partial results (doesn't crash)

Test 18.3 — FTP URL (unsupported scheme)

json

POST /scan

```
{ "url": "ftp://files.example.com" }
```

Expected: API processes it (auto-prefix logic doesn't apply to ftp), results may have errors in redirect/SSL checks but doesn't crash.

Test 18.4 — Unicode domain (IDN)

json

POST /scan

```
{ "url": "https://例え.jp" }
```

Expected: 200, processes without crash.

TEST GROUP 19 — Score Breakdown Verification

Test 19.1 — Verify weights add up correctly

For any scan result, verify:

```
score_breakdown.heuristics = heuristics.heuristic_score × 0.15
score_breakdown.whois      = whois.whois_score          × 0.15
score_breakdown.dns        = dns.dns_score              × 0.08
score_breakdown.ssl        = ssl.ssl_score              × 0.07
score_breakdown.redirects  = redirects.redirect_score   × 0.10
score_breakdown.virustotal = (malicious×20 + suspicious×10) × 0.30
score_breakdown.urlhaus    = urlhaus.urlhaus_score      × 0.07
score_breakdown.phishtank  = phishtank.phishtank_score  × 0.05
score_breakdown.gsb        = gsb.gsb_score              × 0.03
```

`combined_final` may be higher than the raw sum due to critical signal overrides.

Test 19.2 — Critical signals trigger override

```
json

POST /scan
{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

Verify:

- `critical_signals_triggered: 3`
- `critical_sources: ["VirusTotal (malicious≥3)", "Heuristics (score=100.0)", "DNS (score=70.0)"]`
- `combined_final: 90` (override fired, not raw weighted sum)

TEST GROUP 20 — End-to-End Production Simulation

Test 20.1 — Full FYP Demo Sequence

Run these in order to demonstrate all features working together:

1. Safe URL (establishes baseline)

```
json
```

POST /scan

```
{ "url": "https://github.com" }
```

2. Phishing URL (shows detection working)

json

POST /scan

```
{ "url": "http://paypal-secure-verify-account.tk/login/confirm" }
```

3. Bulk scan (shows scale)

json

POST /scan/bulk

```
{  
  "urls": [  
    "https://google.com",  
    "http://paypal-secure-verify-account.tk/login/confirm",  
    "https://github.com"  
  ]  
}
```

4. Async submit + poll (shows non-blocking)

json

POST /scan/async

```
{ "url": "https://claude.ai" }
```

Copy `{job_id}` → `GET /scan/status/{job_id}` after 60s

5. Feedback correction

json


```
POST /feedback
{
  "scan_id": "...",
  "url": "https://github.com",
  "original_risk": "Safe",
  "corrected_risk": "Safe",
  "feedback_type": "correct",
  "confidence_score": 0.8
}
```

6. View metrics

```
GET /metrics
```

7. View feedback stats

```
GET /feedback/stats
```

8. Screenshot proxy

```
GET /proxy-image?url={screenshot_url from step 1}
```

Known Limitations (Document in FYP)

Issue	Cause	Impact
<code>dns_no_mx</code> fires for <code>claude.ai</code>	DNS check penalizes sites with no email MX (product sites often don't have MX)	Minor false flag, doesn't affect classification
<code>ssl_free_ca</code> fires for Let's Encrypt	Free CA penalty designed for phishing sites — also fires for legitimate sites using LE	Minor flag, contributes ~10 SSL score
ML: ~30% phishing probability for <code>claude.ai</code>	Training data lacks examples of legitimate sites using LE + no MX together	Resolved by collecting feedback and retraining
OpenPhish feed ~300 URLs	Public feed is a small sample — commercial feed has 50,000+	Use URL-level checks as primary signal
URLScan screenshots take 60s+	URLScan processes scans asynchronously on their servers	Expected behavior — not a bug

Issue	Cause	Impact
Training accuracy 100% = overfitting	Training on raw URLs only exercises URL-structure features; most features identical defaults	Fixed in v2.0 notebook with proper depth limits